

Member 3 - SoC Integration: Task Sheet

CNN Accelerator with RISC-V SoC Integration | Deadline: 2 weeks (finish by Tue 20 Oct)

1. What changed and why

The course brief requires the accelerator and the CPU to **share a single memory** and to **minimise memory transfer**. Today the CPU pushes every pixel and every weight into the accelerator, resending the weights with every image, so about 2,500 bytes cross the UART per image. The new design:

- **Weights are loaded once at boot** and stay inside the accelerator.
- The image goes into a **shared dual-port memory**. The CPU writes it through port A; the accelerator reads it by itself through port B.
- The CPU only says *where* the image is, *how long* it is, and *start*, then waits for *done*.

2. Agreed interface (do not change without telling the team)

Item	Definition
Image memory (new)	2 KB at 0x2000_0000 (replaces the old write-only pixel port). Two slots: offset 0x000 and 0x400. We use slot 0 only for now.
Port A (CPU)	32-bit, write-only . Word address = bus address bits [10:2]. The byte strobes choose which bytes are written, so one store writes 4 pixels (first pixel in the lowest byte). Reads return 0 but must still complete the handshake.
Port B (accelerator)	8-bit read, 11-bit byte address, 1-cycle read latency. Used only by the image reader (Member 2).
CTRL @ 0x4000_0000 +0x00	Write bit0 = 1: one-cycle accel_start pulse (ignored while busy). Read : bit0 = busy (from the reader), bit1 = done (the existing result-ready flag). The old "enable" meaning is removed.
+0x04 to +0x14	Conv weights, bias, shift: unchanged .
IMG_BASE @ +0x18	Byte offset of the image in the shared memory (11 bits). Use 0x000.
IMG_LEN @ +0x1C	Number of pixels (11 bits). Use 784.
Result @ 0x3000_0000	Unchanged. The result-ready flag is cleared by reading it and by accel_start.

Signals between blocks:

Signal	Width	From	To
accel_start	1	your control registers	image reader, max_pool_2d, dense_layer
img_base / img_len	11 / 11	your control registers	image reader
reader_busy	1	image reader	CTRL bit0
imgb_addr / imgb_rdata	11 / 8	reader / memory port B	memory port B / reader

3. Your tasks, in order

#	Task	Details	Done when	Day
1	Fix firmware.hex format	The Makefile writes one byte per line, but \$readmemh in ram_memory expects one 32-bit word per line. Write 4 bytes per line combined little-endian (%08x).	In simulation, RAM word 0 = the first instruction in the firmware.elf disassembly	1
2	Fix the UART receiver	In uart_peripheral.sv, change rx_bit_cnt from 9 to 10. Currently the start bit is taken as data and data bit 7 as the stop bit, so 'S' (0x53) is never received.	Loopback test passes for all 256 byte values	1
3	SDC file	create_clock -period 20 on CLOCK_50 (50 MHz). Do a full compile of the current design.	Compiles; note Fmax and resources	1-2
4	Smoke-test firmware	Firmware sends "OK" over UART and shows a digit on the 7-segment display.	Works in SoC simulation	2

#	Task	Details	Done when	Day
5	Board bring-up (early)	Program the board: display shows the digit, the laptop receives "OK", UART echo works.	Proven on real hardware	3
6	Top-level rewiring (soc_top.sv)	(a) Delete the old pixel-push logic (pixel_axis_tvalid/tdata from stores to 0x2000_0000). (b) Instantiate image_bram_dp with port A on the bus. Write enable only on a valid write to the 0x2 region; word address = addr[10:2]; strobes = wstrb. (c) Instantiate image_reader , with its output driving slidingWindowAXIBRAM s_axis (including s_axis_tuser). (d) Replace slidingWindowAXI with slidingWindowAXIBRAM (DATA_WIDTH 8, ROW_LENGTH 28, PADDING 0); tie m_axis_tready = 1. (e) Route accel_start to the reader, max_pool_2d and dense_layer.	Compiles in simulation and Quartus	6
7	Bus handshake for 0x2 region	Ready must be a one-cycle pulse per access (same valid && !ready guard as the RAM and UART), so each store is written once. Reads return 0 and still give ready, so the CPU never hangs.	Each store is written exactly once in simulation	6
8	Control registers	Add IMG_BASE and IMG_LEN. Change CTRL to: start pulse on write (only if not busy); busy + done on read. Clear result-ready on accel_start. Keep the weight registers unchanged.	Firmware writes, reads back, starts; busy goes high then low	6
9	System window test	Firmware writes a known pattern to 0x2000_0000 and starts the accelerator. Check the sliding-window output in simulation.	676 correct windows in the full system	7
10	Firmware rewrite (main.c)	Boot: receive conv weights, bias, shift, 1690 dense weights and 10 biases once , and write them. Per image: receive 784 bytes; pack 4 per word; store 196 words to 0x2000_0000; write IMG_BASE = 0 and IMG_LEN = 784; write CTRL = 1; poll CTRL bit1; read 0x3000_0000; show it on the display; send it to the laptop. Measure: rdcycle just before CTRL = 1 and just after done. No drain pixels or tuser in firmware: the reader does both.	Runs in simulation	8
11	send_image.py	Send weights once after connecting, then images only, matching the boot and per-image sequence. Fix the header comment (one result per image, not 676).	Talks correctly to the firmware	8
12	End-to-end simulation (with Member 2)	Real quantized digit from Member 1's quantize.py.	Predicted digit = Python integer model, for 2 digits	9
13	Timing closure	Full compile with the SDC. Report any failing paths to their owner (divider: Member 2; argmax and dense: Member 1).	50 MHz met	10
14	Board demo	Weights load; one image, then many, classified correctly.	Correct digits on display and laptop	11
15	Measurements	Accelerator cycles per image, UART time per image, CPU-only baseline in plain C (same integer maths, simple loops), resources, Fmax, power (Quartus Power Analyzer).	All numbers in one table	12

4. What you need from others

From	What	By
Member 2	image_bram_dp.sv, image_reader.sv, updated slidingWindowAXIBRAM.sv, all tested	Day 5 (Sun 11 Oct)

From	What	By
Member 1	Unsigned pixel fix in mac_unit; accel_start input on max_pool_2d and dense_layer; duplicate mac_unit.sv deleted	Day 3 (Fri 9 Oct)
Member 1	quantize.py: int8 weights, bias, shift + bit-exact integer reference model + export file for the firmware boot phase	Day 6 (Mon 12 Oct)

5. Rules

- Test each step before starting the next one. On the board you cannot see inside the chip.
- Do not change addresses or signal names in Section 2 without telling everyone.
- Commit working steps to GitHub with clear messages, so we can roll back if something breaks.
- If you are blocked for more than half a day, tell the team immediately.